CSA1717-ARTIFICIAL INTELLIGENCE

CO2-AT3-DRY RUN TEST

REPORT

Question 1: A Search Algorithm*

Title

Implementation and Dry Run of A Search Algorithm*

Aim

To implement the A* Search Algorithm to determine the shortest path between a start node and a goal node using heuristic values.

Objective

- To understand the working of the A* Search Algorithm.
- To find the optimal path from the start node to the goal node.
- To evaluate the nodes using the evaluation function **f(n) = g(n) + h(n)**.
- To compare the actual path cost with heuristic estimates.

Problem Statement

Given a weighted graph and heuristic values for each node, implement the A* Search Algorithm to find the shortest path from node **A** to node **G**. Display the current node, open list, closed list, g(n), h(n), and f(n) at every iteration, and determine the optimal path and total path cost.

Algorithm

1. Initialize the Open List with the start node.
2. Set the cost of the start node **g(n)=0**.
3. Compute **f(n)=g(n)+h(n)**.

4. Remove the node with the smallest **f(n)** from the Open List.
5. If the current node is the goal node, stop the search.
6. Otherwise, expand all neighboring nodes.
7. Compute their **g(n)** and **f(n)** values.
8. Add unexplored neighbors to the Open List.
9. Move the current node to the Closed List.
10. Repeat until the goal node is reached or the Open List becomes empty.

Input

- Graph with weighted edges.
- Heuristic values.
- Start Node.
- Goal Node.

Sample Input

Start Node : A

Goal Node : G

Sample Output

Optimal Path : A → B → E → D → G

Total Path Cost : 8

Result

The A* Search Algorithm successfully found the optimal path from **A** to **G** as **A → B → E → D → G** with a minimum total path cost of **8** by evaluating each node using the heuristic function **f(n)=g(n)+h(n)**.

Conclusion

The A* Search Algorithm efficiently determines the shortest path by combining the actual path cost and heuristic estimate. Compared to uninformed search algorithms, A* reduces unnecessary node exploration and guarantees the optimal solution when an admissible heuristic is used.

Question 2: Minimax Algorithm with Alpha-Beta Pruning

Title

Implementation and Dry Run of Minimax Algorithm with Alpha-Beta Pruning

---

Aim

To implement the Minimax Algorithm with Alpha-Beta Pruning for selecting the optimal move in a two-player game tree.

Objective

- To understand the Minimax decision-making process.
- To apply Alpha-Beta Pruning for reducing unnecessary node evaluations.
- To determine the best move for the MAX player.
- To identify the nodes that are pruned during the search.

Problem Statement

Given a game tree with MAX and MIN levels, implement the Minimax Algorithm using Alpha-Beta Pruning. Evaluate the tree from left to right while displaying Alpha (α), Beta (β), selected values, and pruned nodes. Finally, determine the best move and the final minimax value.

Algorithm

1. Start at the root node (MAX).
2. Initialize **Alpha** = **-∞** and **Beta** = **+∞**.

3. Evaluate the left MIN subtree.
4. Update the beta value after every child.
5. Return the minimum value to the MAX node.
6. Update the alpha value at the MAX node.
7. Evaluate the right MIN subtree.
8. If **Beta ≤ Alpha**, prune the remaining child nodes.
9. Return the selected values to the MAX node.
10. Choose the maximum value as the final minimax value.

Input

Leaf node values.

Sample Input

Leaf 1 : 3

Leaf 2 : 5

Leaf 3 : 6

Leaf 4 : 9

Leaf 5 : 1

Leaf 6 : 2

Sample Output

Left MIN Value : 3

Right MIN Value : 1

Final Minimax Value : 3

Best Move : Left Subtree

Pruned Nodes : 2

Result

The Minimax Algorithm with Alpha-Beta Pruning successfully determined the optimal move for the MAX player. The final minimax value obtained was **3**, and the node containing value **2** was pruned, reducing the number of node evaluations.

Conclusion

The Minimax Algorithm guarantees the best possible move for the MAX player assuming optimal play by the opponent. Alpha-Beta Pruning significantly improves efficiency by eliminating branches that cannot affect the final decision, thereby reducing the search space while producing the same optimal result.